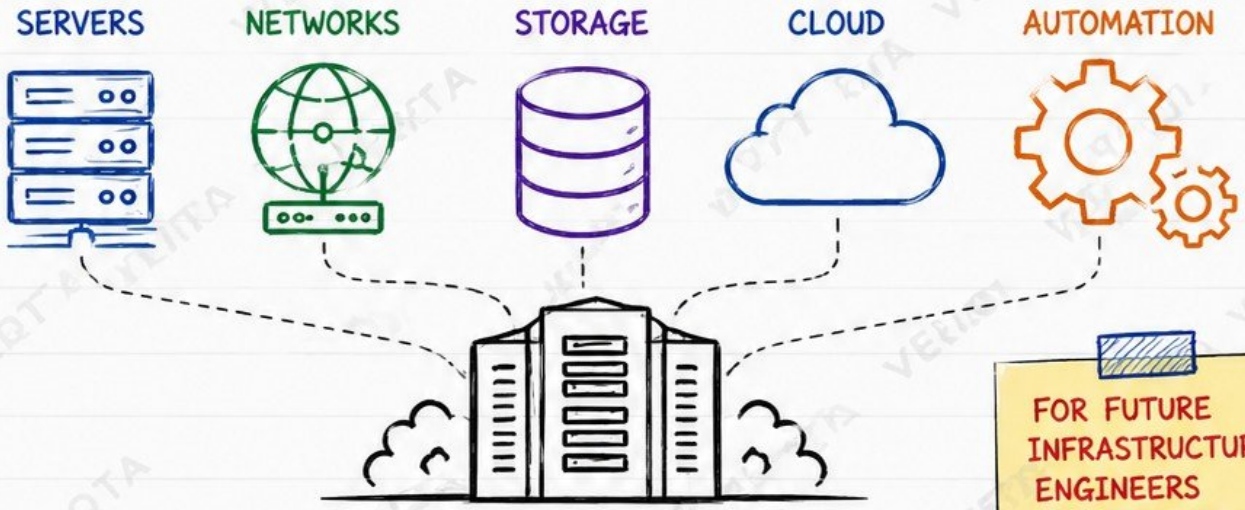
VERIQTA

INFRASTRUCTURE ENGINEERING FOR BEGINNERS

From Servers and Networks to Cloud Infrastructure and Automation



BUILD. CONNECT. SECURE. AUTOMATE. SCALE.

- ✓ Real-World Concepts
- ✓ Hands-On Examples
- ✓ Production Best Practices
- ✓ Modern Tools
- ✓ Step-by-Step Learning
- ✓ Career-Focused

FOR FUTURE INFRASTRUCTURE ENGINEERS

- Understand
- ☑ Build
- ☑ Automate
- ☑ Operate
- ☑ Optimize

YouTube GitHub LinkedIn X Instagram Telegram

VERIQTA

1. WHAT IS INFRASTRUCTURE ENGINEERING?

Build the mental model before touching tools.

1. WHAT INFRASTRUCTURE MEANS IN MODERN ENGINEERING

- Infrastructure is all the systems that run, connect, secure, and store the data for applications.

2. INFRASTRUCTURE ENGINEER RESPONSIBILITIES

- Design, build, deploy, secure, operate, and optimize the systems that power applications and services.

3. CORE BUILDING BLOCKS

- **Compute** (servers, containers),
- **Network** (connectivity, DNS, routing),
- **Storage** (data at rest),
- **Identity** (users, access, permissions).

4. INFRASTRUCTURE TYPES

- **Physical:** Hardware in data centers.
- **Virtual:** VMs and virtual networks.
- **Cloud:** On-demand, scalable resources.

5. INFRASTRUCTURE VS OTHERS

VS DEVOPS	DevOps focuses on delivery and automation. Infrastructure focuses on the foundation.
VS PLATFORM ENGINEERING	Platform Engineering builds internal developer platforms on top of infrastructure.
VS SRE	SRE focuses on reliability and production operations. Infrastructure builds and maintains the systems.

6. INFRASTRUCTURE LIFECYCLE

PROVISION → CONFIGURE → OPERATE → MONITOR → IMPROVE

7. PRODUCTION OWNERSHIP

- Own the reliability, security, performance, and cost of the infrastructure.

- ☑ AVAILABILITY
- ☑ SECURITY
- ☑ PERFORMANCE
- ☑ COST

8. MODERN INFRASTRUCTURE ENGINEER TOOLCHAIN

OPERATING SYSTEMS	VIRTUALIZATION	CLOUD PLATFORMS	IaC	CONFIGURATION MANAGEMENT	CONTAINERS & ORCHESTRATION	MONITORING	OTHER TOOLS
• Linux • Windows	• KVM • VMware • Hyper-V	• AWS • Azure • GCP	• Terraform • CloudFormation • Pulumi	• Ansible • Puppet • Chef	• Docker • Kubernetes	• Prometheus • Grafana • CloudWatch	• Git • CI/CD • DNS • Networking

9. THE INFRASTRUCTURE FLOW

USER → NETWORK → COMPUTE → APPLICATION → DATA

REAL WORLD LESSON:

Infrastructure is the foundation every application depends on.

VERIQTA

2. HOW SERVERS ACTUALLY WORK

Understand the machines behind applications.

1. WHAT A SERVER REALLY IS

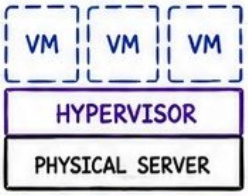
- A server is a computer system that provides resources, services, and data to other systems over a network.

CPU	The brain. Executes instructions and handles all computations.
MEMORY (RAM)	Working space for running programs and processes.
STORAGE	Holds OS, applications, and data (HDD, SSD, NVMe).
NETWORK INTERFACES	Connects the server to networks and other systems.
OPERATING SYSTEM	Manages hardware and provides services to applications.
PROCESSES & & SERVICES	Background or foreground tasks that do real work.

2. SERVER TYPES

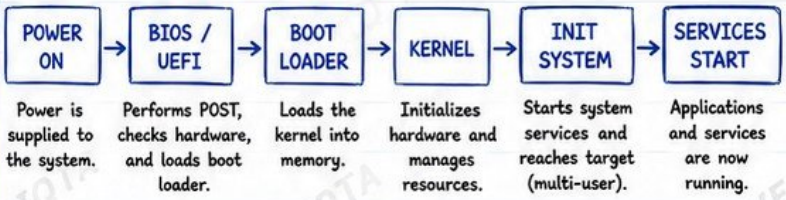
- **BARE-METAL SERVERS**
  Physical servers dedicated to one customer or workload.

- **RACK SERVERS**
  Standardized servers mounted in racks for density, power, cooling, and management.



- **VIRTUAL SERVERS**
  Virtual machines running on physical hardware via a hypervisor.

3. SERVER BOOT PROCESS



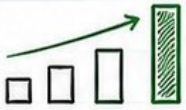
BIOS vs BIOS: Legacy firmware, limited to MBR disks.
- UEFI: Modern firmware, supports GPT, faster boot, secure boot, larger disks.

4. RESOURCE UTILIZATION

CPU SATURATION	CPU at or near 100% for sustained periods.
MEMORY PRESSURE	High memory usage, swapping, or Out Of Memory (OOM) kills.
DISK EXHAUSTION	Disk full or high I/O wait causes slow systems and failures.
HARDWARE FAILURE	Disk, memory, PSU, or NIC can fail without warning.

5. SCALING APPROACHES

VERTICAL SCALING (SCALE UP)



- Add more power to a single server (CPU, RAM, SSD).
- Simple but has hardware limits and single point of failure.

HORIZONTAL SCALING (SCALE OUT)

- Add more servers and distribute the load.
- More resilient and supports massive growth.

PRODUCTION NOTE

A running server is NOT necessarily a healthy server.
Always monitor, measure, and validate before assuming everything is fine.

VERIQTA

3. DATA CENTERS AND PHYSICAL INFRASTRUCTURE

Understand what exists beneath the cloud.

1. DATA CENTER FUNDAMENTALS

A data center is a facility that houses the physical components that power applications and services.

SERVER RACKS
Enclosures that organize and secure servers and equipment.

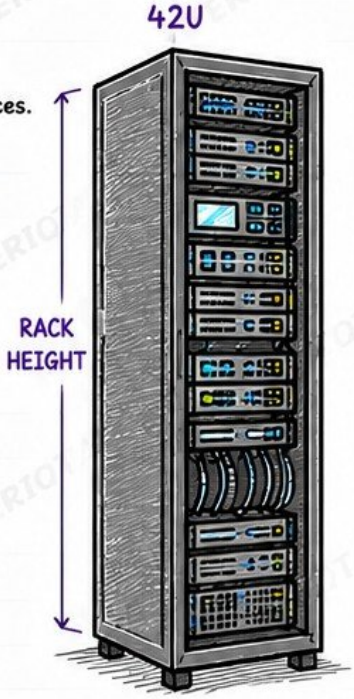
RACK UNITS (U)
Standard height measurement. 1U = 1.75 inches.

POWER DISTRIBUTION
Distributes clean and stable power to all equipment.

UPS SYSTEMS
Provides backup power instantly during outages.

BACKUP GENERATORS
Supplies power during extended outages.

COOLING
Removes heat and maintains safe operating temperatures.



2. NETWORK AND CONNECTIVITY

NETWORK SWITCHES
Connects devices within the data center network.

ROUTERS
Routes traffic between different networks.

FIREWALLS
Protects the network from unauthorized access.

CABLING
Structured cabling ensures speed, reliability, and safety.

TOP-OF-RACK SWITCHES
Connects servers in a rack to the network.

3. HIGH AVAILABILITY DESIGN

- ☑ **REDUNDANT POWER**
  Multiple power paths and sources.
- ☑ **REDUNDANT NETWORKING**
  Multiple switches, routers, and network paths.
- ☑ **NO SINGLE POINT OF FAILURE**
  Every critical component has a backup.

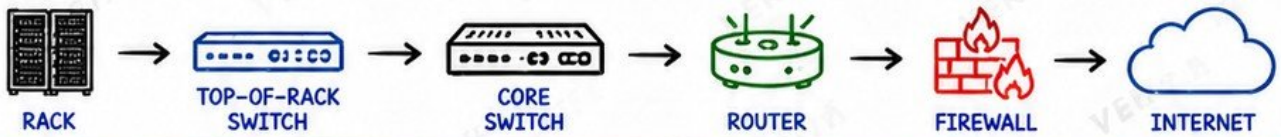
4. RELIABILITY CONCEPTS

- **AVAILABILITY ZONES**
  Physically separate locations within a region.
- **FAILURE DOMAINS**
  Isolated components that do not fail together.
- **DISASTER SCENARIOS**
  Plan for power loss, network failure, hardware failure, fire, flood, and more.

5. WHY CLOUD ENGINEERS SHOULD CARE

- The cloud runs in real data centers.
- Understanding physical infrastructure helps you design for reliability.
- Helps you troubleshoot better.
- Helps you understand limits, failures, and recovery.
- Helps you make better architecture and cost decisions.

6. BASIC NETWORK FLOW



RELIABILITY NOTE
Redundancy must remove single points of failure, not just duplicate them.

VERIQTA

4. VIRTUALIZATION AND VIRTUAL MACHINES

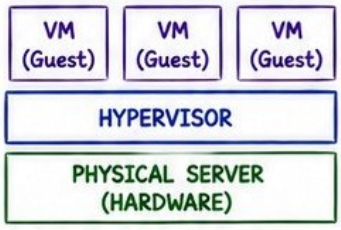
Learn how one physical server becomes many logical servers.

1. WHAT VIRTUALIZATION SOLVES

- Run multiple systems on one physical server.
- Better resource utilization.
- Isolation, flexibility, and easier management.
- Faster provisioning and scaling.

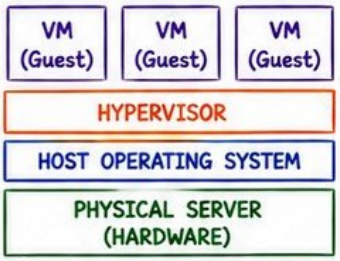
2. HYPERVISORS

TYPE 1 (BARE-METAL)



- Runs directly on hardware.
- More secure and performant.
- Examples: VMware ESXi, KVM, Microsoft Hyper-V.

TYPE 2 (HOSTED)



- Runs on top of a host OS.
- Easier for learning and testing.
- Examples: VirtualBox, VMware Workstation, Parallels.

3. CORE VIRTUAL RESOURCES

VIRTUAL CPUS (vCPU)
Share physical CPU using scheduling.

VIRTUAL MEMORY
Uses host RAM for VM workloads.

VIRTUAL DISKS
Files on storage that act like real disks.

VIRTUAL NICS
Virtual network interfaces connected to virtual switches.

4. VM LIFECYCLE

VM IMAGE
Template used to create VMs.

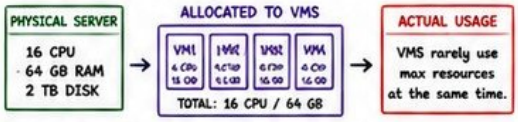
CREATE
Provision a new virtual machine.

RUN
VM is powered on and running.

SNAPSHOT
Save VM state for rollback.

CLONE
Create copies of a VM.

5. RESOURCE OVERCOMMITMENT



Overcommitment increases utilization but can cause contention and performance issues.

6. PLATFORM EXAMPLES

VMWARE CONCEPTS

- ESXi Hypervisor
- vCenter
- vSphere
- VM Templates
- vMotion

KVM CONCEPTS

- Open Source
- Kernel-based
- Uses QEMU
- libvirt & virt-manager
- Strong Performance

VIRTUALBOX

- Type 2 Hypervisor
- Easy to install
- Great for learning
- Supports snapshots
- Lightweight

7. VM VS PHYSICAL SERVER

- VMs share hardware.
- Easier to provision.
- Lower cost.
- Limited by host.

- Dedicated resources.
- Better for high performance.
- Higher cost.
- Hardware-level control.

8. VM VS CONTAINER

- Heavier, full OS per VM.
- Strong isolation.
- Slower to start.
- More resource usage.

- Shares host OS kernel,
- Lightweight.
- Starts in seconds.
- Higher density.

9. PERFORMANCE CONSIDERATIONS

- CPU scheduling and steal time.
- Memory ballooning and swapping.
- Disk I/O contention.
- Network throughput limits.
- Plan capacity and monitor continuously.

10. PRODUCTION FAILURE

RESOURCE CONTENTION BETWEEN VMS

- One noisy VM consumes too many resources.
- Other VMs slow down or become unstable.
- Applications time out or fail.
- Users experience high latency.

REAL WORLD LESSON: Virtualization gives you flexibility, but you must design for isolation, monitoring, and capacity. Overcommit early, but know your limits.

VERIQTA

5. LINUX FOR INFRASTRUCTURE ENGINEERS

Make Linux your primary infrastructure workstation.

1. WHY LINUX DOMINATES INFRASTRUCTURE

- Open source, secure, and stable.
- Runs servers, clouds, containers, and networks.
- Highly customizable and scriptable.
- Huge community and enterprise support.

2. LINUX FILESYSTEM HIERARCHY

/etc	System configuration files.
/var	Variable data: logs, caches, spools, databases.
/home	Home directories for users.
/tmp	Temporary files. Cleared on reboot.
/proc	Virtual filesystem with process and kernel info.
/sys	Virtual filesystem for kernel and device info.
/usr	User binaries and applications.
/bin	Essential user binaries.
/sbin	System administration binaries.
/var/log	System and application logs.

3. USERS AND GROUPS

- Users own files and processes.
- Groups manage access and permissions.

```
# add user
sudo useradd engineer
# set password
sudo passwd engineer
# add to group
sudo usermod -aG wheel engineer
```

4. FILE PERMISSIONS

Each file has: owner, group, others.

Permissions: read (r) write (w) execute (x)

```
-rw-r--r--  1 engineer  wheel  1024 May 20 file.txt
            ↑          ↑      ↑
          owner      group  others
```

5. PROCESSES

- Programs in execution.

```
ps aux
top
htop
```

6. SERVICES (DAEMONS)

- Run in background.

```
systemctl status nginx
systemctl start nginx
systemctl enable nginx
```

7. SYSTEMD BASICS

- systemd is PID 1 and manages the system.
- Unit types: service, socket, timer, target, mount.

```
systemctl list-units --type=service
systemctl status sshd
systemctl enable httpd
systemctl restart docker
```

8. JOURNALCTL (LOGS)

- View and filter systemd logs easily.

```
journalctl -xe
journalctl -u nginx
journalctl --since "1 hour ago"
```

9. PACKAGE MANAGEMENT

DISTRO	PACKAGE MANAGER
DEBIAN / UBUNTU	apt / apt-get
RHEL / CENTOS / ROCKY	yum / dnf
SUSE	zypper
ARCH LINUX	pacman

EXAMPLES

```
sudo apt update && sudo apt upgrade
sudo dnf install nginx
sudo yum install httpd
```

10. ENVIRONMENT VARIABLES

- Store system-wide or user-specific configuration.

```
echo $PATH
export APP_ENV=production
env | sort
```

11. SSH (SECURE ACCESS)

- Use SSH keys, not passwords.
- Default port: 22

```
ssh user@server
ssh-copy-id user@server
```

12. LOGS

- Logs tell you what happened.
- Always check logs first.

```
less /var/log/syslog
less /var/log/messages
tail -f /var/log/nginx/access.log
```

13. BASIC RESOURCE TROUBLESHOOTING

CPU	top, htop, mpstat
MEMORY	free -h, vmstat, swapon --show
DISK SPACE	df -h, du -sh *
DISK I/O	iostat -x, iotop
NETWORK	ss -tulpen, netstat -s
LOAD AVERAGE	uptime

SENIOR ENGINEER TIP

Learn to inspect a system before changing it.
Observe first, understand second, act last.

INSPECT → UNDERSTAND → ACT

VERIQTA

6. NETWORKING FUNDAMENTALS

Understand how infrastructure communicates.

1. NETWORK FUNDAMENTALS

- Networks connect devices to share data and resources.
- LAN: Local Area Network (within a building or campus).
- WAN: Wide Area Network (across cities or the internet).
- Infrastructure relies on reliable and fast networking.

2. IP ADDRESSES

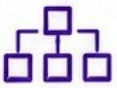
- Unique logical address for every device.
- IPv4: 32-bit address (example: 192.168.1.10).
- IPv6: 128-bit address (example: 2001:db8::1).

3. PRIVATE vs PUBLIC IPs

10.0.0.0/8
172.16.0.0/12
192.168.0.0/16

8.8.8.8
1.1.1.1
203.0.113.10

4. SUBNETS & CIDR



- Subnetting divides a network into smaller networks.
- CIDR notation represents networks efficiently.
- Example: 192.168.1.0/24 (256 total addresses).

5. DEFAULT GATEWAYS

- Gateway is the exit point to another network.
- Sends traffic to destinations outside the local network.
- Example: 192.168.1.1

6. MAC ADDRESSES

AA:BB:CC:DD:EE:FF

- Unique hardware address of a network interface.
- Used at Layer 2 (Data Link).
- Cannot be changed (usually).

7. ARP (ADDRESS RESOLUTION PROTOCOL)

- Maps IP address to MAC address on the local network.

8. ROUTERS

- Connect different networks.
- Make decisions based on IP addresses (Layer 3).

9. SWITCHES

- Connect devices within the same network.
- Forward traffic using MAC addresses (Layer 2).

10. TCP vs UDP

TCP

- ✓ Connection-oriented
- ✓ Reliable
- ✓ Ordered delivery
- ✓ Used by: HTTP, HTTPS, SSH, FTP

UDP

- ✓ Connectionless
- ✓ Faster
- ✓ No delivery guarantees
- ✓ Used by: DNS, DHCP, Streaming, VoIP

11. PORTS

- Identify applications on a device.
- Range: 0 – 65535
- Examples: 22 (SSH), 80 (HTTP), 443 (HTTPS), 53 (DNS)

12. DNS

- Translates domain names to IP addresses.
- Example: veriqta.com → 172.67.1.10

13. DHCP

- Automatically assigns IP addresses and settings.
- Saves time and reduces manual errors.

14. NAT (NETWORK ADDRESS TRANSLATION)

- Allows multiple private hosts to share one public IP.
- Used in home networks and cloud environments.

15. PACKET FLOW (HOW DATA TRAVELS)

Data is broken into packets, routed across networks, and reassembled at the destination.

16. REQUEST FLOW DIAGRAM

CLIENT — User device

DNS — Resolves domain to IP address

ROUTER — Routes packets to destination network

SERVER — Processes request and sends response

CLIENT — Receives response and displays data

DEBUGGING INSIGHT: TROUBLESHOOT NETWORKING LAYER BY LAYER

1. PHYSICAL LAYER — Check cables, links, and hardware.
2. NETWORK LAYER — Check IP, routes, subnets, gateways.
3. TRANSPORT LAYER — Check ports, TCP/UDP, firewall rules.
4. APPLICATION LAYER — Check DNS, services, and application logs.

VERIQTA

7. SUBNETTING, ROUTING, AND NETWORK DESIGN

Move from networking concepts to infrastructure design.

1. NETWORK ADDRESS vs HOST ADDRESSES

192 . 168 . 1 . 0 / 24

NETWORK ADDRESS
Identifies the network.

HOST ADDRESSES
Used by devices within the network.

2. CIDR NOTATION

- CIDR (Classless Inter-Domain Routing) replaces classful addressing.
- Example: 10.0.0.0/24
- /24 means 24 bits for network, 8 bits for hosts.

More bits for network = more subnets, fewer hosts.

3. SUBNET MASKS

CIDR	SUBNET MASKS	USABLE HOSTS
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62
/27	255.255.255.224	30
/28	255.255.255.240	14

4. WHY SUBNETTING EXISTS

- Efficient use of IP addresses.
- Improve security by isolating networks.
- Reduce broadcast domains.
- Better performance and organization.

5. PUBLIC vs PRIVATE SUBNETS

PUBLIC SUBNETS
- Routable on the internet.
- Have route to Internet Gateway.

PRIVATE SUBNETS
- Not directly reachable from the internet.
- Use NAT Gateway for outbound access.

6. ROUTING TABLES

DESTINATION	TARGET	TYPE
10.0.0.0/16	local	Local
0.0.0.0/0	igw-123456	Internet
0.0.0.0/0	nat-abcdef	NAT

Routes tell the network where to send traffic.

7. DEFAULT ROUTES

- 0.0.0.0/0 means "anywhere".
- Used for internet or outbound traffic.

8. INTERNET GATEWAYS

- Attach to VPC.
- Enable communication between VPC and Internet.

9. NAT GATEWAYS

- Provide outbound internet access for private subnets.
- Hide private IPs.

10. NETWORK SEGMENTATION

- Break networks into smaller parts.
- Limit access and blast radius.

11. NORTH–SOUTH vs EAST–WEST TRAFFIC

NORTH–SOUTH TRAFFIC (INTERNET ↔ VPC)

VPC

- Enters or leaves the VPC.
- Controlled by IGW, NAT, Firewall, LB.

EAST–WEST TRAFFIC (WITHIN VPC)

- Traffic between resources inside the VPC.
- Controlled by Security Groups, NACLs.

12. MULTI-TIER NETWORK DESIGN

WEB TIER
Public Subnet
↓
APPLICATION TIER
Private Subnet
↓
DATABASE TIER
Private Subnet

- Separates responsibilities.
- Improves security and manageability.
- Supports scalability and high availability.

13. WEB → APP → DB SEGMENTATION

WEB (PUBLIC) → APP (PRIVATE) → DB (PRIVATE)

- Only necessary traffic is allowed.
- Web tier → App tier (allowed)
- App tier → DB tier (allowed)
- No direct access to DB from Internet.

14. ROUTE SELECTION (LONGEST PREFIX MATCH)

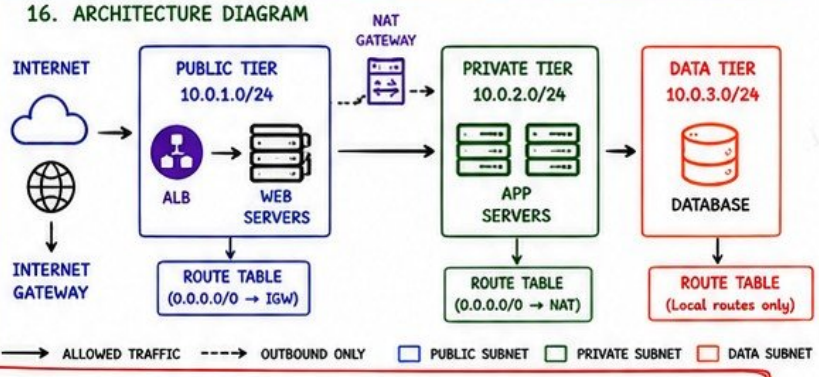
Router chooses the most specific route.

DESTINATION	CIDR	TARGET
10.1.1.0	/24	local
10.1.0.0	/16	nat-123456
0.0.0.0	/0	igw-123456

15. OVERLAPPING CIDRS

- Overlapping CIDRs cause routing confusion.
- Avoid using the same IP ranges across networks.

16. ARCHITECTURE DIAGRAM



SECURITY REMINDER

Do not expose infrastructure simply because routing allows it.
Always control access with security groups, NACLs, and firewalls.

VERIQTA

8. STORAGE FUNDAMENTALS

Understand where infrastructure keeps data.

1. STORAGE TYPES

BLOCK STORAGE
Raw blocks. Used by servers and databases.
Example: EBS, iSCSI.

FILE STORAGE
Shared files over network.
Used by many systems.
Example: NFS, SMB.

OBJECT STORAGE
Stores objects in buckets.
Great for scale and durability.
Example: S3, GCS.

2. LOCAL vs NETWORK STORAGE

LOCAL STORAGE
- Attached directly to the server.
- Faster, lower latency.
- Not shared.
- Example: Local SSD, NVMe.

NETWORK STORAGE
- Accessed over network.
- Shareable and scalable.
- Slightly higher latency.
- Example: EBS, NFS.

3. MEDIA COMPARISON

	HDD	SSD
SPEED	Slower	Much Faster
IOPS	Lower	Much Higher
LATENCY	Higher	Much Lower
THROUGHPUT	Lower	Higher
COST	Cheaper	Expensive
USE CASE	Cold Data, Backups	Databases, Hot Data

4. KEY STORAGE METRICS

- **IOPS** (Input/Output Operations Per Second): How many operations the storage can handle.
- **THROUGHPUT**: Amount of data moved per second (MB/s or GB/s).
- **LATENCY**: Time taken to complete an operation (measured in ms).

High IOPS + High Throughput + Low Latency = High Performance Storage

5. STORAGE STRUCTURE

- **FILESYSTEM** — Organizes data (ext4, xfs, btrfs).
- **MOUNT POINT** — Where filesystem is attached in OS (/data, /var).
- **VOLUME** — Logical block device created from disk(s).
- **DISK / DEVICE** — Physical storage (HDD, SSD, NVMe).

6. RAID CONCEPTS

RAID

- **RAID 0**: Striping (Performance)
- **RAID 1**: Mirroring (Redundancy)
- **RAID 5**: Striping + Parity (Balanced)
- **RAID 10**: Mirror + Stripe (High Perf. + Redundancy)

7. DATA PROTECTION

SNAPSHOTS → **BACKUPS** → **REPLICATION** → **FAILOVER**

- **SNAPSHOTS**: Point-in-time copy. Fast and space-efficient.
- **BACKUPS**: Copy stored separately. For long-term recovery.
- **REPLICATION**: Data copied to another location for HA/DR.
- **FAILOVER**: Automatic or manual switch to healthy copy.

8. STORAGE CLASSIFICATION

PERSISTENT STORAGE
- Survives system rebuilds.
- Used for databases, applications, user data.

EPHEMERAL STORAGE
- Temporary and volatile.
- Used for caches, logs, scratch data.

9. CAPACITY PLANNING

- Understand current usage and growth trend.
- Plan for peak usage, not average.
- Leave buffer for headroom (20–30%).
- Monitor and review regularly.
- Plan for backups and replication overhead.

10. STORAGE FAILURE MODES

- Disk failure
- Controller failure
- Network failure
- Data corruption
- Human error
- Full disk / No space

PERFORMANCE TIP
Storage bottlenecks can look like application problems.

Always check:
- ☑ IOPS
- ☑ Latency
- ☑ Throughput
- ☑ Disk saturation

STORAGE IS NOT JUST DISK.
IT IS PERFORMANCE, RELIABILITY, AND DATA PROTECTION.

VERIQTA

9. DNS, LOAD BALANCING, AND TRAFFIC FLOW

Understand how users reach applications.

1. DNS RESOLUTION

1. User enters example.com
2. DNS resolver checks cache
3. If not found, it queries DNS servers
4. Returns IP address
5. Browser connects to the IP

2. DNS RECORDS

A	Maps domain to IPv4 address
AAAA	Maps domain to IPv6 address
CNAME	Alias of another domain name
TTL	Time to live. How long the record is cached
Recursive Resolution	Resolver does the full lookup on your behalf

3. LOAD BALANCERS

- Distribute traffic across multiple servers.
- Improve availability and performance.
- Provide a single entry point for applications.

LAYER 4	LAYER 7
• Transport layer (TCP/UDP) • Uses IP + Port • Fast and simple • No content inspection	• Application layer (HTTP/HTTPS) • Uses host, path, headers, cookies • Smarter routing • Supports URL rules

4. LOAD BALANCER FEATURES

HEALTH CHECKS
Periodically checks if a server is healthy.

BACKEND POOLS
Group of servers behind the load balancer.

REVERSE PROXIES
Accept client requests and forward to servers.

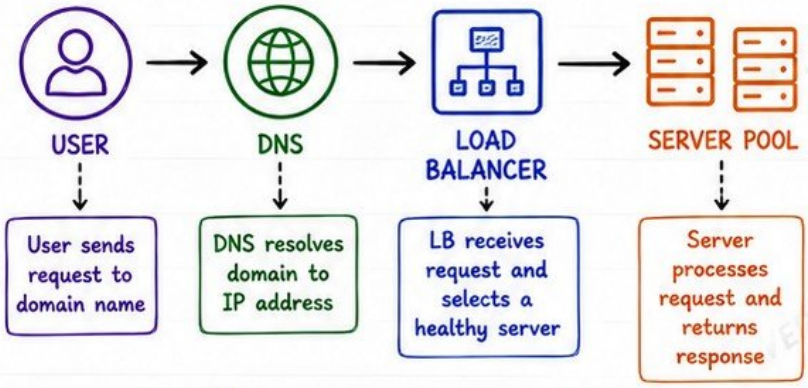
TLS TERMINATION
Decrypt HTTPS at the load balancer, forward HTTP to backend.

SESSION PERSISTENCE
Send a user's requests to the same server.

HIGH AVAILABILITY
LBs run across multiple nodes / AZs.

TRAFFIC DISTRIBUTION
Round robin, least connections, weighted, IP hash, etc.

5. REQUEST FLOW



HOW LB SELECTS A SERVER
- Check server health
- Apply routing algorithm
- Forward request
- Return response to user

COMMON ALGORITHMS
- Round Robin
- Least Connections
- Weighted
- IP Hash

PRODUCTION FAILURE

Healthy servers can still be unreachable through a broken traffic path. →

- ✕ DNS misconfiguration
- ✕ Expired TLS certificate
- ✕ Load balancer health check misconfiguration
- ✕ Security groups / firewall blocking traffic
- ✕ NAT / routing issues

VERIQTA

10. INFRASTRUCTURE SECURITY FUNDAMENTALS

Secure infrastructure from the beginning.

1. AUTHENTICATION VS AUTHORIZATION

AUTHENTICATION
Verifies who you are.

- Example: Password, SSH key, MFA.

AUTHORIZATION
Determines what you can do.

- Example: IAM role, permissions, policies.

2. IDENTITY & ACCESS MANAGEMENT (IAM)

- Manage users, groups, roles, and policies.
- Centralize access control.
- Use federated identity when possible.

3. LEAST PRIVILEGE

- Grant only the minimum access required.
- Reduces blast radius.
- Review and rotate permissions regularly.

4. SSH SECURITY

- Disable password auth.
- Change default port.
- Limit access by IP.
- Keep SSH server updated.

5. SSH KEYS

- Use key-based authentication.
- Protect private keys.
- Use passphrases.
- Never share keys.

6. FIREWALLS

- Control inbound and outbound traffic.
- Default deny, allow only what is needed.

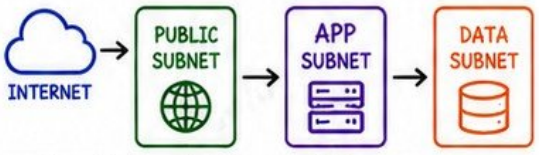
7. SECURITY GROUPS

- Stateful firewall at the instance level.
- Allow by port, protocol, and source.

8. NETWORK ACLs

- Stateless firewall at the subnet level.
- Rule number order matters.

9. NETWORK SEGMENTATION



- Separate tiers (public, app, data).
- Limit lateral movement.
- Use private subnets for sensitive resources.

10. ADMINISTRATIVE ACCESS

- Restrict admin access.
- Use IAM roles, not long-term keys.
- Enforce MFA.

11. BASTION HOSTS

- Secure entry point into private networks.
- No direct SSH to private instances.
- Log and monitor all access.

12. SECRETS

- Never hardcode secrets.
- Use a secrets manager.
- Rotate secrets regularly.
- Limit access to secrets.

13. ENCRYPTION AT REST

- Encrypt disks, databases, and backups.
- Protect data if storage is stolen.
- Use KMS or equivalent.

14. ENCRYPTION IN TRANSIT

- Encrypt data while moving over networks.
- Prevent eavesdropping.

15. TLS

- Use TLS for all web traffic.
- Use strong ciphers.
- Keep certificates valid and updated.

16. PATCH MANAGEMENT

- Keep OS and software updated.
- Automate patching.
- Test before production.

17. VULNERABILITY MANAGEMENT

- Scan regularly.
- Prioritize based on risk.
- Remediate quickly.

18. HARDENING

- Disable unused services.
- Enforce secure configs.
- Follow CIS benchmarks.
- Remove unnecessary software.

19. AUDIT LOGS

- Log all key events.
- Monitor and alert on anomalies.
- Store logs securely.

20. SECURITY REMINDER

Private-by-default is safer than expose-then-restrict.

VERIQTA

11. INTRODUCTION TO CLOUD INFRASTRUCTURE

Translate traditional infrastructure into cloud services.

1. WHAT CLOUD COMPUTING ACTUALLY CHANGES

- You stop buying hardware.
- You use resources as services.
- You pay for what you use.
- You gain elasticity and speed.
- You focus on building, not maintaining.
- You shift from managing infrastructure to designing systems.

2. CLOUD SERVICE MODELS

IaaS	**INFRASTRUCTURE AS A SERVICE** You manage OS, middleware, runtime, apps, and data. Provider manages compute, storage, networking.
PaaS	**PLATFORM AS A SERVICE** You deploy and manage your apps. Provider manages OS, runtime, middleware, and infrastructure.
SaaS	**SOFTWARE AS A SERVICE** You use the application. Provider manages everything including the application.

3. CORE CLOUD BUILDING BLOCKS

REGIONS Geographic areas.	**AVAILABILITY ZONES** Isolated data centers in a region.	**COMPUTE** Virtual servers, containers, functions.
NETWORKING VPCs, subnets, routing, load balancers.	**STORAGE** Object, block, file storage services.	**DATABASES** Managed SQL, NoSQL, caching, and more.
IDENTITY Users, roles, permissions, access control.	**MANAGED SERVICES** Pre-built services you do not have to operate.	**ELASTICITY** Grow or shrink resources on demand.

SCALABILITY Handle more load by scaling out or up.	**HIGH AVAILABILITY** Design across AZs to stay available during failures.

4. SHARED RESPONSIBILITY MODEL

YOU MANAGE (IN THE CLOUD)	CLOUD PROVIDER MANAGES
• Data • Applications • Operating Systems • Network Configurations • Access Management • Identity and Permissions	• Physical Facilities • Hardware • Networking Infrastructure • Virtualization Layer • Core Services • Power, Cooling, Security of Data Centers

5. CLOUD PROVIDERS: CORE CONCEPTS

AWS Regions, AZs, VPC, IAM, EC2, S3, RDS, ELB, CloudWatch	**AZURE** Regions, AZs, VNet, Axure AD, VM, Blob Storage, SQL Database, Monitor	**GOOGLE CLOUD** Regions, Zones, VPC, IAM, Compute Engine, Cloud Storage, Cloud SQL, Cloud Ops

6. DEPLOYMENT MODELS

ON-PREMISES
You own and operate everything.

CLOUD
Provider owns and operates.

HYBRID
You operate some, provider operates some.

7. ON-PREMISES VS CLOUD

ON-PREMISES	CLOUD
High upfront cost	Pay as you go
Limited by hardware	Elastic and scalable
You maintain everything	Provider manages much of it
Slow to provision	Fast to provision
Capacity planning required	Scale automatically

ENTERPRISE ADVICE
Cloud removes hardware ownership, not infrastructure responsibility. You still own availability, security, performance, and architecture.

12. BUILDING INFRASTRUCTURE IN AWS

Connect infrastructure concepts to a real cloud architecture.

1. KEY AWS SERVICES AND CONCEPTS

AWS REGIONS Geographic locations worldwide.	**AVAILABILITY ZONES** Isolated data centers within a region.	**VPC** Your isolated virtual network.	**SUBNETS** Divide your VPC into smaller networks.	**ROUTE TABLES** Control where network traffic goes.	**INTERNET GATEWAY** Connects your VPC to the internet.	**NAT GATEWAY** Provides outbound internet access for private subnets.
EC2 Virtual servers in the cloud.	**EBS** Persistent block storage for EC2.	**S3** Object storage service.	**ELASTIC LOAD BALANCING** Distributes traffic across multiple targets.	**ROUTE 53** DNS and domain management.	**IAM** Manage users, roles, and permissions.	**SECURITY GROUPS** Virtual firewalls for your resources.
CLOUDWATCH Monitoring, logs, and alerts.	**RDS** Managed databases in the cloud.	**MULTI-AZ THINKING** Design for high availability across AZs.				

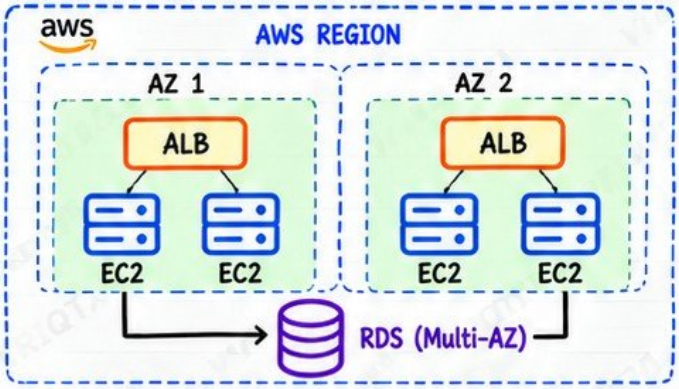
WHY THESE SERVICES MATTER

- They replace traditional infrastructure.
- They work together to build secure, scalable, highly available systems.
- They give you the building blocks for real production environments.

2. REFERENCE ARCHITECTURE: ROUTE 53 → ALB → EC2 → RDS

USERS Access your application. → **ROUTE 53** DNS routes traffic to ALB. → **ALB** Distributes traffic across EC2 instances. → **EC2 (AUTO SCALING)** Application servers in private subnets. → **RDS (MULTI-AZ)** Managed database with high availability.

3. MULTI-AZ ARCHITECTURE (HIGH LEVEL)



4. KEY BENEFITS

High availability	Survives instance or AZ failures.
Scalability	Scale based on demand.
Security	Use VPC, subnets, and security groups.
Reliability	Managed services with built-in resilience.
Cost efficiency	Pay for what you use.

PRODUCTION NOTE:
Design around failure domains, not individual resources.

13. HIGH AVAILABILITY AND FAULT TOLERANCE

Design infrastructure that survives failures.

1. WHAT AVAILABILITY MEANS

- The ability of a system to be ready for use when needed.
- Measured as a percentage (e.g., 99.9% = "three nines").

2. KEY PRINCIPLES

- **SINGLE POINTS OF FAILURE:** Any component whose failure stops the system.
- **REDUNDANCY:** Duplicate critical components so another can take over.
- **FAULT DOMAINS:** Physical or logical boundaries where failures can occur independently.
- **AVAILABILITY ZONES (AZs):** Isolated data centers within a region.

3. DEPLOYMENT STRATEGIES

ACTIVE-ACTIVE
Traffic is served by all instances or AZs.

ACTIVE-PASSIVE
Standby takes over when active fails.

ACTIVE ---> STANDBY

4. HIGH AVAILABILITY MECHANISMS

- **LOAD BALANCING:** Distributes traffic across healthy instances.
- **HEALTH CHECKS:** Continuously verify if instances are healthy.
- **AUTOMATIC FAILOVER:** Traffic is moved to healthy components automatically.

5. STATEFUL VS STATELESS

STATELESS SERVICES

No user/session data on the instance.

Easy to scale and replace.

STATEFUL SERVICES

Store user data or session information.

Require persistent storage and careful design.

6. DATA & DATABASE RESILIENCE

- **DATABASE REDUNDANCY:** Deploy across multiple AZs.
- **REPLICATION:** Keep copies of data in real time.
- **READ REPLICAS:** Offload read traffic and improve performance.

7. RECOVERY OBJECTIVES

RTO (RECOVERY TIME OBJECTIVE)

How fast your system must be restored after a failure.

Example: RTO = 15 minutes

RPO (RECOVERY POINT OBJECTIVE)

How much data loss is acceptable after a failure.

Example: RPO = 5 minutes

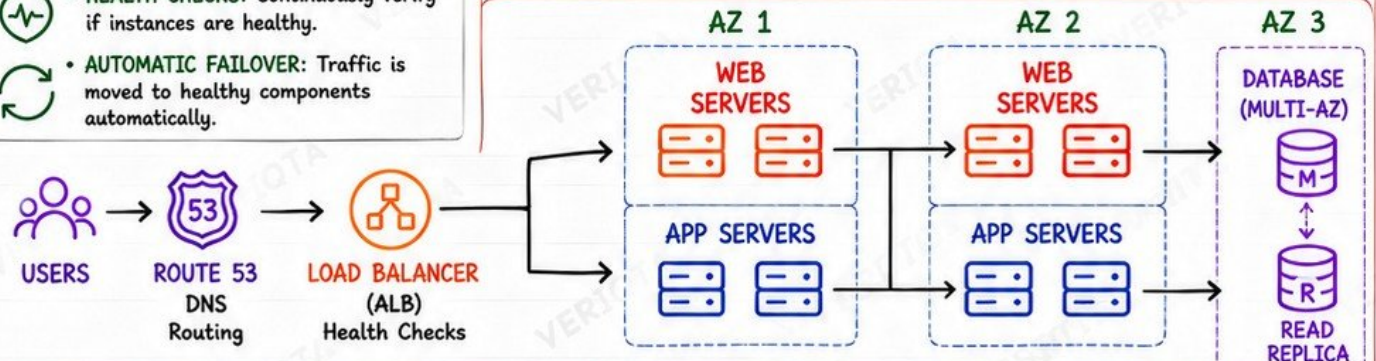
8. GRACEFUL DEGRADATION

When part of the system fails, the application continues to work with reduced features instead of complete failure.

9. RELIABILITY CHECKLIST

- ☑ No single point of failure
- ☑ Multi-AZ deployment
- ☑ Load balancer + health checks
- ☑ Automatic failover enabled
- ☑ Data replicated and backed up
- ☑ RTO and RPO defined
- ☑ Graceful degradation in place

10. MULTI-AZ APPLICATION STACK ARCHITECTURE



REDUNDANCY	AUTO FAILOVER	SCALE OUT	BACKUPS
Across AZs	Traffic moves automatically	Add more instances	Protect your data

RELIABILITY LESSON:
Assume components will fail. Build systems that expect failure, detect it quickly, and recover automatically.

14. INFRASTRUCTURE AS CODE

Stop building infrastructure manually.

1. WHAT INFRASTRUCTURE AS CODE SOLVES

- Eliminates manual, error-prone work.
- Makes infrastructure repeatable.
- Enables versioning, review, and audit.
- Speeds up provisioning and changes.
- Allows safe experimentation.
- Ensures consistency across environments.

2. APPROACHES

IMPERATIVE	DECLARATIVE
• You define steps. • Focus on how to do it. • Example: Scripts.	• You define the desired state. • Focus on what you want. • Example: Terraform.

IaC uses DECLARATIVE approach.

3. DESIRED STATE

You describe the infrastructure as it should be.
The tool figures out how to create or update it.

4. TERRAFORM FUNDAMENTALS

TERRAFORM

- Open source IaC tool by HashiCorp.
- Works with multiple cloud providers.
- Uses HCL (HashiCorp Configuration Language).
- Plans changes before applying.
- Tracks state of your infrastructure.

5. CORE BUILDING BLOCKS

PROVIDERS	Connect Terraform to cloud or services.
RESOURCES	Define infrastructure components.
VARIABLES	Make configurations dynamic and reusable.
OUTPUTS	Expose important values after deployment.
STATE	Stores the current state of your infrastructure.
MODULES	Reusable infrastructure components.

6. TERRAFORM COMMANDS

PLAN	Preview changes before applying.
APPLY	Create or update infrastructure.
DESTROY	Remove infrastructure.

7. BEST PRACTICES

- Version control everything.
- Review changes before apply.
- Use modules for reuse.
- Separate environments.
- Keep state secure.
- Detect and fix drift.
- Automate with CI/CD.
- Document your IaC.

8. IAC WORKFLOW

GIT — Code is stored in version control.
→ **REVIEW** — Team reviews the changes.
→ **PLAN** — See what will be changed.
→ **APPLY** — Changes are applied to infrastructure.
→ **INFRASTRUCTURE** — Your infrastructure is updated.

9. ENVIRONMENT SEPARATION

DEV	STAGING	PRODUCTION
For development and testing.	For pre-production validation.	For live workloads.

10. DRIFT

Drift happens when real infrastructure changes outside of IaC.
Detect drift and bring it back to the desired state.

11. REMOTE STATE

Store state in a remote backend (e.g., S3, DynamoDB).
Enables team collaboration and prevents conflicts.

PRODUCTION NOTE:
Infrastructure changes should be reviewable and reproducible.

15. CONFIGURATION MANAGEMENT AND AUTOMATION

Automate what happens after infrastructure exists.

1. PROVISIONING VS CONFIGURATION

PROVISIONING

Create and prepare infrastructure resources (servers, networks, storage, etc.).

CONFIGURATION

Install, configure, and manage software and services on the infrastructure.

Provisioning builds it. Configuration makes it work.

2. CONFIGURATION MANAGEMENT

Ensures systems are configured consistently, securely, and repeatedly.

- Reduces manual work
- Enforces standards
- Improves security
- Ensures consistency
- Makes systems repeatable
- Detects and fixes drift

3. ANSIBLE FUNDAMENTALS

INVENTORY

List of hosts or groups Ansible manages.

PLAYBOOKS

YAML files that define automation steps.

TASKS

Individual steps executed on target systems.

MODULES

Pre-built units that perform specific actions.

VARIABLES

Dynamic values used in playbooks and templates.

TEMPLATES

Files with placeholders that render dynamic configurations.

ROLES

Reusable collections of tasks, files, templates, and variables.

IDEMPOTENCY

Running a playbook multiple times gives the same result without side effects.

SSH-BASED AUTOMATION

Ansible connects to targets via SSH. Agentless by default.

CONFIGURATION DRIFT

When systems deviate from the desired configuration.

4. COMMON AUTOMATION TASKS

- Package installation
- Service configuration and management
- User and group management
- File and directory management
- Permission and ownership management
- System updates and patching
- Security hardening
- Log management

5. TERRAFORM VS ANSIBLE

TERRAFORM

- Infrastructure as Code
- Provisioning
- Manages cloud resources
- Declarative
- Uses providers

ANSIBLE

- Configuration as Code
- Post-provisioning
- Manages servers and services
- Procedural (YAML)
- Uses modules

Terraform builds infrastructure. Ansible configures it.

6. AUTOMATION FLOW

TERRAFORM → Provision infrastructure (IaC).

SERVERS → New or existing servers are ready.

ANSIBLE → Connects via SSH and runs playbooks on servers.

CONFIGURED SYSTEMS → Systems are now configured, secured, and consistent.

7. SENIOR ENGINEER TIP

Automate repeatable operations, not broken processes.

Fix the process first. Then automate it.

16. CONTAINERS AND INFRASTRUCTURE

Understand how containers change infrastructure.

1. CONTAINERS

A container packages an application and all its dependencies so it runs consistently anywhere.

2. IMAGES

A read-only template used to create containers.
Built with instructions (Dockerfile).

3. REGISTRIES

Stores and distributes container images.
Examples:
Docker Hub, ECR, GHCR, ACR.

4. DOCKER

The most popular container platform for building, shipping, and running containers.

5. CONTAINER RUNTIME

Software that runs containers on the host.
Examples: containerd, CRI-O, Docker Engine.

6. HOW CONTAINER ISOLATION WORKS

NAMESPACES

Isolate container resources so each container feels like it has its own system.

- PID Namespace
- Network Namespace
- Mount Namespace
- IPC Namespace
- UTS Namespace

CGROUPS

Limit and control container resource usage.

- CPU
- Memory
- Disk I/O
- Network
- PIDs

RESULT

Containers are isolated from each other and from the host.

- Lightweight
- Secure by default
- Efficient resource usage

7. CONTAINER NETWORKING

- Each container gets its own network interface.
- Containers on the same network can communicate.
- Bridge, host, overlay networks.
- Ports expose services to the outside.

8. CONTAINER STORAGE

- Containers are ephemeral.
- Data must be persisted using volumes or bind mounts.
- Volumes survive container restarts and recreation.

9. ESSENTIAL CONTAINER CONCEPTS

PORTS

Expose container services to other containers or to the outside world.

VOLUMES

Persist data outside the container lifecycle.

ENVIRONMENT VARIABLES

Configure apps dynamically without changing images.

CONTAINER LIFECYCLE

Create → Start → Stop → Restart → Remove.
Containers are not pets, they are cattle.

10. VM vs CONTAINER

VIRTUAL MACHINES		CONTAINERS
Full OS per VM	VS	Share host OS kernel
Heavyweight		Lightweight
Slower to start		Fast to start
Higher resource usage		Lower resource usage

11. INFRASTRUCTURE IMPLICATIONS

- Higher density on the same hardware
- Faster deployments and rollbacks
- Consistent environments across lifecycle
- Requires new thinking for networking, storage, and security
- Stateless by default
- Ephemeral compute
- Built for automation
- Observability becomes critical

12. IMMUTABLE INFRASTRUCTURE

Containers should be treated as immutable artifacts.
Build once, test once, run anywhere.
Do not change containers after deployment.

13. WHY ORCHESTRATION BECOMES NECESSARY

- Containers are ephemeral and can fail anytime.
- You need automated scheduling, scaling, healing, and service discovery.
- Orchestrators (like Kubernetes) handle:
  - Placement • Scaling • Self-healing • Rolling updates
  - Networking • Storage • Secrets • ConfigMaps

14. PRODUCTION FAILURE

TREATING CONTAINERS LIKE PERMANENT VIRTUAL MACHINES

- Installing software inside running containers
- Storing data in the container filesystem
- SSH into containers and making changes
- No version control for container images
- No automation or orchestration

This leads to drift, failures, slow deployments, and unreliable systems.

15. CONTAINER ECOSYSTEM OVERVIEW

16. REAL WORLD LESSON

Containers change how we build, deploy, and operate infrastructure.
Embrace automation, immutability, and orchestration to build reliable, scalable systems.

VERIQTA

17. KUBERNETES INFRASTRUCTURE FUNDAMENTALS

Understand the infrastructure underneath Kubernetes.

1. WHY KUBERNETES EXISTS

- Containers need orchestration.
- Kubernetes automates deployment, scaling, healing, and operations.
- It provides a consistent platform to run containerized applications.

2. CLUSTER ARCHITECTURE OVERVIEW

A Kubernetes cluster has two main parts:

- Control Plane – brain of the cluster
- Worker Nodes – run your workloads

3. CONTROL PLANE COMPONENTS

API SERVER
- Front door to the cluster. All requests go here.

ETCD
- Distributed key-value store. Holds all cluster data.

SCHEDULER
- Decides which node runs each Pod.

CONTROLLER MANAGER
- Runs controllers that keep the cluster in desired state.

4. WORKER NODE COMPONENTS

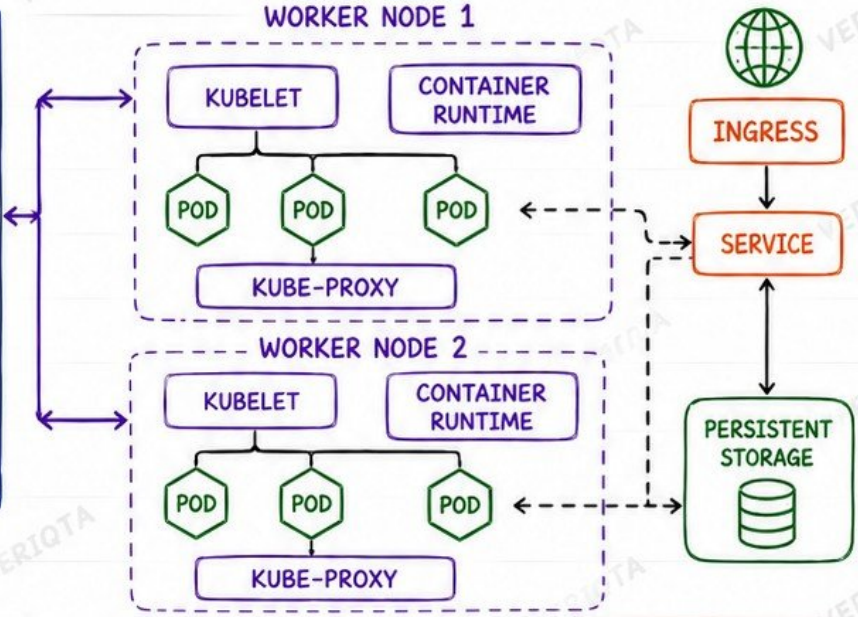
KUBELET
- Agent that ensures containers are running in Pods.

CONTAINER RUNTIME
- Runs containers (e.g., containerd, CRI-O).

KUBE-PROXY
- Handles Service networking and load balancing.

5. CORE KUBERNETES OBJECTS

- PODS – Smallest deployable unit.
- SERVICES – Stable networking for Pods.
- INGRESS – HTTP/HTTPS access into the cluster.
- PERSISTENT STORAGE – Keeps data beyond Pod lifecycle.
- CLUSTER NETWORKING – Allows Pods to communicate across nodes.
- NODE CAPACITY – CPU, Memory, Pods, and Storage limits.



7. INFRASTRUCTURE RESPONSIBILITIES

- Design the cluster architecture.
- Choose instance types and node sizing.
- Plan network, storage, and security.
- Ensure high availability and backups.
- Monitor, optimize, and scale the cluster.
- Manage upgrades and infrastructure lifecycle.

8. MANAGED KUBERNETES SERVICES

Cloud providers manage the control plane for you.

- AMAZON EKS (AWS)
- MICROSOFT AKS (AZURE)
- GOOGLE GKE (GCP)

You still manage nodes, networking, storage, security, and workloads.

SCALING NOTE

Kubernetes does not eliminate infrastructure management.

It abstracts complexity, but the responsibility remains yours.

REAL WORLD LESSON: Kubernetes is a powerful orchestration platform, but a healthy cluster depends on strong infrastructure.

VERIQTA

18. MONITORING AND OBSERVABILITY

Know when infrastructure is unhealthy.

1. MONITORING VS OBSERVABILITY

MONITORING	OBSERVABILITY
• Tells you WHAT is happening.	• Helps you understand WHY it is happening.
• Uses known metrics, logs, and alerts.	• Uses metrics, logs, traces, and context.
• Answers: Is the system healthy?	• Answers: Why is the system unhealthy?

2. KEY SIGNALS TO WATCH

- CPU UTILIZATION
- MEMORY UTILIZATION
- DISK CAPACITY
- DISK I/O
- NETWORK THROUGHPUT
- NETWORK ERRORS
- LATENCY
- AVAILABILITY
- SATURATION

3. OBSERVABILITY PILLARS

METRICS	LOGS	TRACES
Numbers over time. Shows volume, rate, and usage.	Events with context. Shows what happened.	Requests end-to-end. Shows how things are connected.

4. POPULAR TOOLS

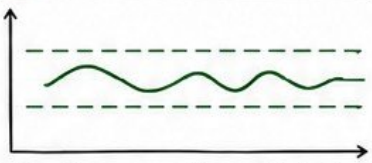
PROMETHEUS	GRAFANA	CLOUDWATCH
• Metrics collection	• Dashboards	• Metrics & Logs
• Time-series DB	• Visualizations	• Alarms
• Alert rules	• Alerts	• Dashboards

5. ALERTING AND DASHBOARDS

ALERTING
- Detect issues early
- Notify the right people
- Reduce MTTR

DASHBOARDS
- Visualize system health
- Track KPIs
- Spot trends fast

6. BASELINES



- Baselines represent normal behavior.
- Alerts trigger on deviations.
- Essential for accurate alerting.

7. SYMPTOMS VS CAUSES

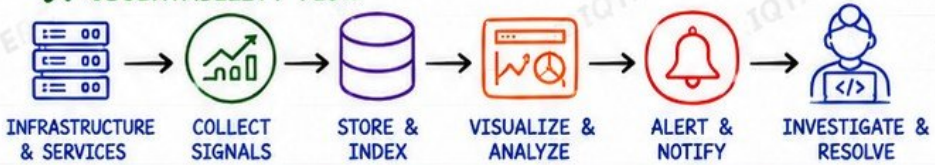
SYMPTOMS	CAUSES
• High latency	• CPU saturation
• 5xx errors	• Memory pressure
• Slow responses	• Disk I/O wait
• Service down	• Network issues
	• Dependency failure

8. DEBUGGING INSIGHT

START WITH SIGNALS, THEN NARROW THE FAILURE DOMAIN.

- Collect signals
- Correlate them
- Form a hypothesis
- Test and confirm
- Fix the real cause

9. OBSERVABILITY FLOW



REAL WORLD LESSON: You can't fix what you can't see. Observability turns unknowns into answers.

VERIQTA

19. PRODUCTION INFRASTRUCTURE TROUBLESHOOTING

Learn a repeatable troubleshooting method.

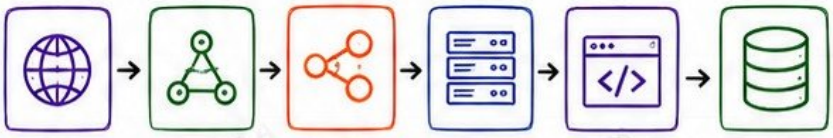
1. THE TROUBLESHOOTING METHOD

1. OBSERVE
2. SCOPE
3. IDENTIFY IMPACT
4. CHECK RECENT CHANGES
5. VERIFY DEPENDENCIES
6. INSPECT METRICS
7. INSPECT LOGS
8. CHECK COMPUTE
9. CHECK MEMORY
10. CHECK STORAGE
11. CHECK NETWORK
12. CHECK DNS
13. CHECK LOAD BALANCERS
14. CHECK PERMISSIONS
15. CHECK CERTIFICATES
16. TEST HYPOTHESES
17. MITIGATE SAFELY
18. VERIFY RECOVERY
19. ROOT CAUSE ANALYSIS
20. PREVENT RECURRENCE

2. SCENARIO

SCENARIO:
Users report:
"Application unavailable"

3. TROUBLESHOOTING FLOW



CHECK IN ORDER. DON'T SKIP LAYERS.

- Failures can happen anywhere in the stack.
- Verify each layer before moving to the next.
- The earlier the failure, the wider the impact.

4. WHAT TO LOOK FOR AT EACH LAYER

DNS	• Can clients resolve the domain? • Correct records and TTL?
NETWORK	• Connectivity, routing, firewall rules? • Packet loss or high latency?
LOAD BALANCER	• Health checks passing? • Backend targets healthy?
COMPUTE	• Instance/Pod running? • High CPU, memory, or restarts?
APPLICATION	• Errors, exceptions, timeouts? • Dependencies responding?
DATABASE	• DB reachable? • Connections, locks, slow queries?

5. QUICK TIPS

- ✓ Start with facts, not assumptions.
- ✓ Check recent changes early.
- ✓ Use metrics and logs, not guesses.
- ✓ Narrow the failure domain.
- ✓ Document what you tried.
- ✓ Communicate impact clearly.

SLOW IS THE NEW DOWN!

6. OUTCOME GOALS

FIND THE ROOT CAUSE → RESTORE SERVICE SAFELY → LEARN AND DOCUMENT → PREVENT IT FROM HAPPENING AGAIN

DEBUGGING INSIGHT

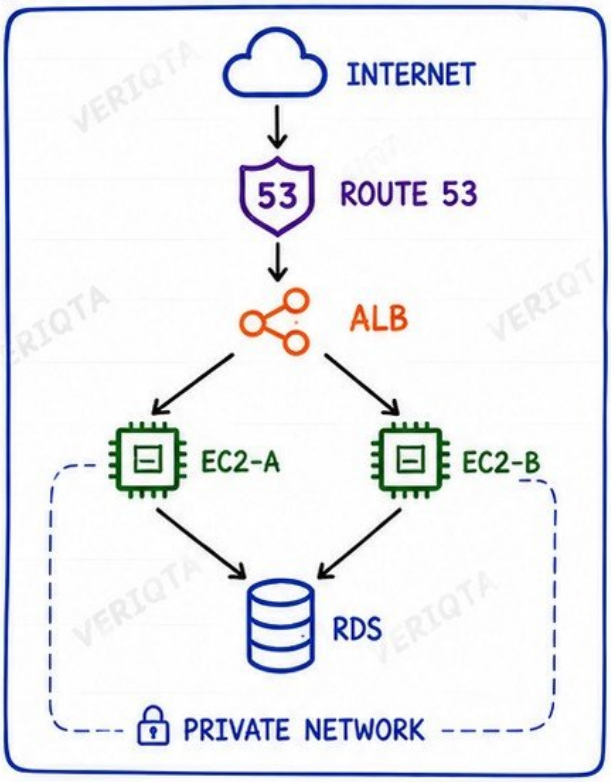
Start with signals, then narrow the failure domain.

VERIQTA

20. BUILD YOUR FIRST PRODUCTION-STYLE INFRASTRUCTURE

Bring the entire handbook together.

1. ARCHITECTURE TO BUILD



2. INFRASTRUCTURE REQUIREMENTS

- ✓ VPC
- ✓ PUBLIC AND PRIVATE SUBNETS
- ✓ MULTI-AZ DESIGN
- ✓ ROUTE TABLES
- ✓ INTERNET CONNECTIVITY
- ✓ CONTROLLED OUTBOUND ACCESS
- ✓ SECURITY GROUPS
- ✓ EC2
- ✓ LOAD BALANCER
- ✓ RDS
- ✓ IAM
- ✓ DNS
- ✓ TLS
- ✓ TERRAFORM PROVISIONING
- ✓ ANSIBLE CONFIGURATION
- ✓ MONITORING AND ALERTS
- ✓ BACKUP STRATEGY
- ✓ FAILURE TESTING
- ✓ ARCHITECTURE DOCUMENTATION

3. FINAL ENGINEERING EXERCISE

FINAL ENGINEERING EXERCISE:

Take one component offline and explain:

- WHAT HAPPENS?
- HOW DOES THE SYSTEM DETECT THE FAILURE?
- HOW DOES TRAFFIC RESPOND?
- HOW DO YOU RECOVER?
- WHAT WOULD YOU REDESIGN?

4. DESIGN PRINCIPLES

- ✓ HIGH AVAILABILITY (MULTI-AZ)
- ✓ SECURE BY DEFAULT
- ✓ LEAST PRIVILEGE ACCESS
- ✓ SCALABLE AND RESILIENT
- ✓ OBSERVABLE AND ALERTED
- ✓ AUTOMATED AND REPEATABLE
- ✓ BACKED UP AND RECOVERABLE
- ✓ COST OPTIMIZED

5. FAILURE SCENARIOS TO TEST

COMPONENT FAILURES
- EC2 INSTANCE
- ALB
- RDS

NETWORK FAILURES
- INTERNET
- SUBNET
- ROUTE

SECURITY FAILURES
- SG RULES
- IAM ROLES
- CERTIFICATES

STORAGE FAILURES
- DISK FULL
- RDS FAILURE
- SNAPSHOT LOSS

PRODUCTION MINDSET:

DO NOT ASK ONLY, "DOES IT WORK?"
ASK, "HOW DOES IT FAIL, HOW WILL I KNOW, AND HOW WILL IT RECOVER?"